Mining Prevalent Co-location Patterns with Multiple Minimum Prevalence Thresholds

Vanha Tran(✉), Thiloan Bui, Thaigiang Do, and Hoangan Le

FPT University, Hanoi 155514, Vietnam
{hatv14,loanbt7,giangdt26}@fe.edu.vn, anlhhe163613@fpt.edu.vn

Abstract. Discovering a set of spatial features from spatial datasets whose instances frequently appear together in the neighbor area of each other called prevalent co-location pattern (PCP) mining is an important technique in data mining. Currently, most mining algorithms use a single minimum prevalence threshold to filter PCPs. However, the number of instances of each feature in datasets varies greatly, and each feature's participation ratio in a pattern is also very different. When a single unified threshold is used, valuable patterns will likely be filtered out. To overcome this shortcoming, this paper proposes a method that uses multiple minimum prevalence thresholds, in which each feature is assigned a suitable threshold according to its number of instances and the application of users. The prevalence threshold of each pattern is dynamic according to the different features participating in the pattern. To mine multiple minimum prevalence threshold-based PCPs efficiently, this paper also designs an algorithm based on a participating instance hash table. First, the dataset is scanned once to enumerate all maximal cliques that are sets of neighboring instances, and then these cliques are organized into a hash structure. Next, the mining process uses these keys of the hash structure as the initial candidate set. The participating instances of each candidate are collected by directly executing a query in the hash structure. When the prevalence of this candidate is judged, its direct subsets are generated as new candidates and put into the candidate set. The proposed method is experimentally tested on synthetic and real datasets in various aspects. The experimental results show that the proposed method is effective and efficient when compared with the state-of-the-art algorithms.

Keywords: Prevalent co-location patterns · Multiple thresholds · Maximal cliques · Level-wise candidate search · Downward closure property

1 Introduction

Nowadays with the development of positioning systems, a large amount of data with coordinate (location) information is generated and stored every day which is called spatial data. How to find valuable knowledge from the massive amount

S. Wu et al. (Eds.): PKAW 2024, LNAI 15372, pp. 1–14, 2025.
https://doi.org/10.1007/978-981-96-0026-7_1

of spatial data is extremely important. It can be said that we are drowning in data, but we are poor in knowledge. Many data mining techniques have been proposed, such as clustering, classification, outlier detection, frequent patterns and association rules, etc. Among them, prevalent co-location patterns (PCPs) refer to finding a set of spatial features from a spatial dataset whose instances frequently appear together in a form of cliques, which is one of the most critical spatial mining technologies. This technology has been applied to many fields such as transportation [12], public safety [4], business [9], etc.

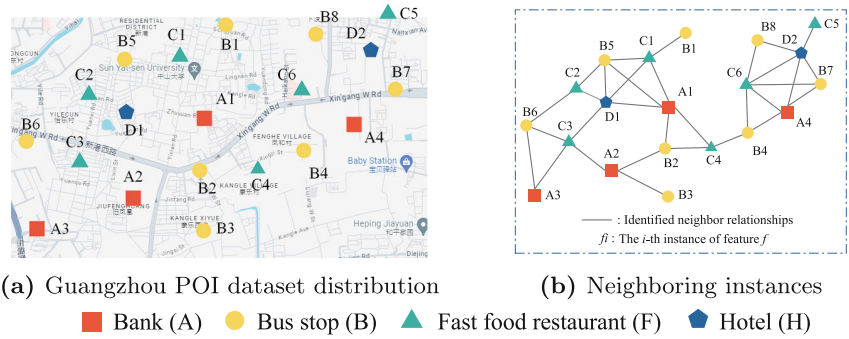


Fig. 1. An illustration of a spatial dataset for PCP mining.

Figure 1 depicts the distribution of a point of interest (POI) dataset of Guangzhou, China [10]. The data has 20 objects, e.g., A2, B1, and so on (called **instances**) that belong to four categories (called spatial **features**), namely, banks (A), bus stops (B), fast food restaurants (F), and hotels (H). A user sets a distance threshold d (e.g., $d = 300$ m) to determine the neighbor relationships between instances, that is, when the distance between two instances is not greater than d, they are neighbors. As shown in Fig. 1(b), instances that satisfy d are connected by a solid line. When applying the PCP mining technique, considering the combination {A,B,C}, its **co-location instances** (groups of instances that have the neighbor relationships with each other) are {A1,B2,C4}, {A3,B6,C3}, and {A4,B4,C6}, the participation ratios (given in Definition 2 later) of the features of the group are $\frac{3}{4}$, $\frac{3}{8}$, $\frac{3}{6}$, respectively, where the denominator of each term is the total number of instances of each feature in the dataset. Thus the prevalence degree (defined by a participation ratio, PI, metric that is given in Definitions 3–4) of this feature group is $PI(\{\text{A,B,C}\}) = \min\{\frac{3}{4}, \frac{3}{8}, \frac{3}{6}\} = 0.375$. The user sets a minimum prevalence threshold μ to filter out his/her interesting patterns. For example, $\mu = 0.2$, since $PI(\{\text{A,B,C}\}) \geq \mu = 0.2$. Therefore, {A,B,C}, i.e., {Banks, Bus stops, Fast food restaurants} is a PCP. Since there are three distinguishing features in this pattern, it is called a size 3 PCP.

The example in Fig. 1 is only a small part of the complete POI dataset. The actual dataset contains many instances, and the number of instances belonging to each feature varies greatly. Figure 2 counts the number of instances under

each feature of the California POI dataset[1]. It can be seen that the number of instances between features varies greatly. The traditional PCP mining method only uses a unique minimum prevalence threshold for all features of all patterns, which easily leads to the loss of some interesting patterns. Therefore, this paper designs a PCP mining algorithm using multiple prevalence thresholds to solve this problem. The specific contributions of this paper are as follows:

(1) Finding PCPs by multiple minimum prevalence thresholds is proposed. Each feature is assigned a minimum prevalence threshold according to its own number of instances, and then the threshold for a pattern is automatically determined according to the features contained in the pattern.
(2) An algorithm that is based on a participating instance hash table is designed to discover multiple minimum prevalence threshold-based PCPs efficiently. This algorithm includes two phases. First, neighboring instances are arranged into a hash table structure based a set of maximal cliques that are enumerated by scanning only once the dataset. Second, all the participating instances of the features in each pattern are quickly obtained by a query operation. Thus, the prevalence of a candidate is judged efficiently.
(3) The effectiveness and efficiency of the designed algorithm are evaluated on comprehensive experiments on both synthetic and real datasets.

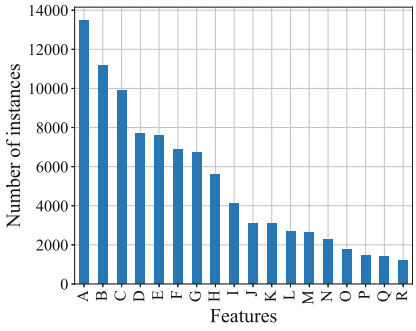


Fig. 2. Numbers of instances of features in the POI dataset of California, USA.

The rest of this paper is organized as follows. Section 2 quickly reviews related work. Relevant definitions and the proposed method are presented in detail in Sect. 3. Section 4 shows and analyzes the experimental results. Finally, Sect. 5 gives conclusions and future work directions.

2 Related Work

Spatial PCPs can be seen as an extension of the famous frequent itemset mining [2], but the former is much more complicated than the latter because there

[1] https://users.cs.utah.edu/~lifeifei/SpatialDataset.htm.

are complex and large numbers of neighbor relationships between instances in spatial data. Huang et al. first proposed the concept of spatial PCP mining and designed an algorithm named Joinbase [5]. This algorithm uses a join operator between the co-location instances of size k PCPs to generate and validate the co-location instances of size $(k+1)$ candidate patterns. The join operation becomes very time-consuming when processing large spatial data sets. To overcome this shortcoming, many excellent algorithms have been proposed later, such as Joinless [15], instance driver scheme (IDS) [1], overlapping cliques [10], clique candidate-based [14], and so on.

Usually, the prevalence threshold value is set as small as possible to avoid filtering out interesting patterns. However, this setting causes the mining results to contain too many patterns, making it difficult for users to understand and apply. Therefore, the concept of simplified patterns is proposed, including maximal PCPs [9], closed PCPs [14], meta-PCPs [8], non-redundant PCPs [11], etc.

Parallel PCP mining algorithms have been proposed to solve the problem of mining of PCPs on large-scale spatial data sets. These parallel algorithms mainly implement the existing algorithms on different parallel platforms such as Hadoop [7], Map-reduce [13], GPU [6], and so on.

All the algorithms mentioned above use a unique minimum prevalence threshold to filter out prevalent patterns. As analyzed in Sect. 1, interesting patterns my be lost due to the imbalance in the number of instances between features. In the following sections, we will introduce the proposed method in detail to overcome this shortcoming.

3 The Proposed Algorithms

3.1 Basic Concepts

Given a set of spatial objects, $DS = \{o_1, \ldots, o_n\}$ (called a spatial dataset), where each object is a triple in the form of $o_i =< f, ID, (x, y) >$, in which the first, second, and third elements represent the feature type, identification, and location of the object, respectively, users set a distance threshold d to materialize the neighbor relationships between instances according to their application requirements, if the distance between different feature instances o_i and o_j is not larger than d, the two instances have a neighbor relationship and denoted as $R(o_i, o_j)$. The feature set of all instances is $F = \{f_1, \ldots, f_m\}$.

Definition 1 (Candidate co-location pattern). *A subset of the feature set, $c = \{f_1, .., f_k\} \subseteq F$, is called a candidate pattern. The number of the feature in the pattern, k, is called the size of the pattern, i.e., c is a size k candidate co-location pattern.*

Definition 2 (Co-location instance, table instance, and participating instance). *A co-location instance of a candidate $c = \{f_1, \ldots, f_k\}$ is a set of instances $CI = \{o_1, \ldots, o_k\}$, that CI included the instances of all features of c and these instances have the neighbor relationship with each other. The set of all*

co-location instances of c is call the table instance and denoted as $Tab(c)$. *The participating instances of a feature* $f_t \in c$ *is the set of distinguishing instances of* f_t *that appear in the table instance of c, i.e.,* $PartI(f_t, c) = \{o_i | o_i \in Tab(c) \wedge$ *feature of* o_i *is* $f_t\}$.

For example, consider a candidate $c = \{A,B,C\}$ the dataset shown in Fig. 1(b), all co-location instances of it are {A4, B7, C6}, {A4, B4, C6}, {A1, B2, C4}, {A1, B5, C1}, and {A3, B6, C3}. Thus, the participating instances of features A, B, and C respectively are $PartI(A, c) = \{A1, A3, A4\}$, $PartI(B, c) = \{B2, B4, B5, B6, B7\}$, and $PartI(C, c) = \{C1, C3, C4, C6\}$.

Definition 3 (Participation ratio and participation index (PI)). *The participation ratio of a feature* $f_t \in c = \{f_1, \ldots, f_k\}$ *is denoted as* $PR(f_t, c) = \frac{|PartI(f_t,c)|}{|I_{f_t}|}$ *where* $|I_{f_t}|$ *is the cardinality of the set of instances that belong to feature* f_t. *The participation index of c is the minimum participation ratio, i.e.,* $PI(c) = min_{\forall f_t \in c}\{PR(f_t, c)\}$.

Definition 4 (Prevalent co-location pattern (PCP)). *Users set a minimum prevalence threshold* μ, *if the PI of a candidate is not smaller than the threshold, i.e.,* $PI(c) \geq \mu$, *c is a PCP.*

Continuing the above example, we have $PR(A, \{A, B, C\}) = \frac{3}{4}$, $PR(A, \{A, B, C\}) = \frac{3}{8}$, and $PR(C,\{A, B, C\}) = \frac{3}{6}$, where 4, 8, 6 are the total numbers of instances of features A, B, and C, respectively. Thus, $PI(\{A, B, C\}) = \min\{\frac{3}{4}, \frac{3}{8}, \frac{3}{6}\} = 0.375$. If users set $\mu = 0.2$, since $PI(\{A, B, C\}) > \mu$, {A, B, C} is a PCP.

3.2 The Multiple Minimum Prevalence Thresholds

Since the number of instances of each feature in datasets varies greatly, the number of instances of each feature participating in a pattern is also very different, and the instances of the same feature participating in different patterns are also different, when a single prevalence threshold is used, some valuable patterns are likely to be filtered out. Thus, the method proposed in this paper is that users set a minimum prevalence threshold for each feature according to their own application and the threshold of a pattern is the minimum feature threshold of these features participating in the pattern. The threshold of a pattern is dynamically adapted according to the participating features involved in it.

Definition 5 (Feature threshold). *Each feature* f_t *in the dataset is assigned a minimum prevalence threshold* $\mu(f_t)$. $\mu F = \{\mu(f_1), \ldots, \mu(f_m)\}$ *is the set of thresholds for all features in the dataset.*

For example, a user sets the minimum frequent threshold for each feature in the data set shown in Fig. 1 as $\mu(A) = 0.3$, $\mu(B) = 0.6$, $\mu(C) = 0.45$, and $\mu(D) = 0.15$.

Definition 6 (Pattern threshold). *Given a pattern* $c = \{f_1, \ldots, f_m\}$*, the minimum prevalence threshold of the pattern is the minimum feature threshold involved in the pattern, that is,*

$$\mu(c) = min\{\mu(f_1), ..., \mu(f_k)\} \tag{1}$$

If the PI of c is larger than the threshold, $PI(c) \geq \mu(c)$*, c is a PCP.*

For example, consider pattern {A,B,C}, its minimum prevalence threshold is $\mu(\{A, B, C\}) = \min\{\mu(A), \mu(B), \mu(C)\} = \min\{0.3, 0.6, 0.45\} = 0.3$. Since $PI(\{A, B, C\})$ ¿ $\mu(\{A, B, C\})$, thus {A, B, C} is a PCP.

3.3 The Designed Mining Algorithm

From Definitions 2 to 4 and the running example, we can easily find that the most critical phase in calculating the PI of a pattern is how to collect the participating instances of each feature involved in the pattern, $PartI(f_t, c)$. It is based on co-location instances, which means that we have to first find all co-location instances belonging to the pattern. However, when datasets are dense, collecting co-location instances becomes very expensive. If we can directly find $PartI(f_t, c)$ and avoid collecting co-location instances, the mining algorithm can be improved.

Definition 7 (Undirected graph). *When the neighbor relationships between instances are materialized, an undirected graph is obtained,* $G(V, E)$*, where the vertices are instances, i.e.,* $V \equiv DS$ *and the edge set, E, is the set of the lines between two instances that satisfy the neighbor relationship.*

Definition 8 (Maximal clique, MC). *A clique* $cl = \{o_1, ..., o_l\}$ *is a set of instances in which each instance has the neighbor relationship with each other. If there is not any cliques* cl' *such that* $cl \subset cl'$*, cl is a maximal clique.*

Our work here does not focus on designing a new MC enumeration algorithm, but on how to use MCs to improve the mining of PCPs. Although enumerating MCs is an NP-hard problem, it has attracted a lot of researchers, and many enumerating MC algorithms have been proposed. We chose Bron-Kerbosch with the degeneracy as the algorithm for finding MCs in this work. Due to the limitation of the margin, the details of it are not expanded here, please refer to [3].

Table 1 lists all MCs that are enumerated from the dataset after materializing neighbor relationships between instances shown in Fig. 1(b).

Table 1. Enumerated MCs from Fig. 1(b)

MCs	MCs	MCs	MCs	MCs
{A1, B5, C1, D1}	{A2, B3}	{A4, B7, C6, D2}	{B5, C2, D1}	{C3, D1}
{A1, B2, C4}	{A2, C3}	{B1, C1}	{B6, C2}	{C5, D2}
{A2, B2}	{A3, B6, C3}	{B4, C4}	{B8, C6, D2}	

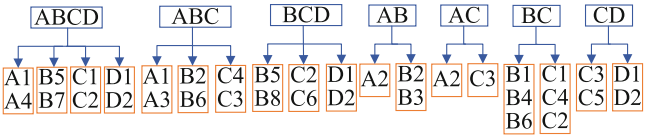


Fig. 3. The constructed participating instance hash table.

Definition 9 (Participating instance hash table). *The participating instance hash table is a hash table structure where the keys are the feature sets of the instances in the maximal cliques and the values are the instances of these maximal cliques.*

Figure 3 shows the hash table structure that is constructed by Definition 9 based on the maximal cliques listed in Table 1. Next, all the participating instances of any patterns can be easily obtained from the hash structure through a query operator.

Lemma 1. *Given a candidate pattern* $c = \{f_1, \ldots, f_k\}$*, the participating instances of each feature involved in* c *consist of the values in the hash table whose keys are a superset of* c*.*

Proof. Since the hash table is composed of MCs, if a key is a superset of a pattern $c = \{f_1, \ldots, f_k\}$, it means that the value of this key is a MC and this MC is also a superset of a co-location instance of c. c has multiple keys that are supersets at the same time, thus collecting the values corresponding to these supersets together constitutes a complete participating instance set of the features in c.

For example, consider $c = \{\text{A, B, C}\}$, from the hash table, its superset keys are ABCD and ABC, thus $PartI(\text{A}, c) = \{\text{A1, A4}\} \cup \{\text{A1, A3}\} = \{\text{A1, A3, A4}\}$, where {A1, A4} is queried form key ABCD and {A1 ,A3} is obtained from key ABC.

Algorithm 1 is the pseudo-code of the designed mining PCP algorithm using multiple prevalence thresholds (MPCP-MPT for short). First, the neighbor relationship between instances is materialized under the threshold d set by the user (step 1), and then the MCs are enumerated using the Bron-Kerbosch with the degeneracy algorithm (step 2). Note that in this step, to optimize the efficiency, the MCs are not stored separately here, each time a MC is found, it is directly placed in the hash structure. Next, all the keys in the hash table are used as the

Algorithm 1: **M**ining **PCP**s with **M**ultiple **P**revalence **T**hreshold algorithm (MPCP-MPT for short)

Input: $DS, d, \mu F$
Output: $PCPs$: a set of prevalent co-location patterns

```
1  G(V,E) ← materialize_neighbor_relationship(DS, d)
2  Hash ← enumerate_MCs_and_build_Hash(DS, G(V,E))
3  Cands ← initial_candidates(Hash)
4  while Cands ≠ ∅ do
5  |  c ← Cands.pop() ;        /* nonPCPs save non-prevalent patterns */
6  |  if c ∉ PCPs and c ∉ nonPCPs then
7  |  |  PartIs ← query_participating_instances(c, Hash)
8  |  |  UPI ← compute_PI(PartIs, c)
9  |  |  μc ← compute_threshold(c, μF);
10 |  |  if UPI ≥ μc then
11 |  |  |  PCPs ← PCPs ∪ {c}
12 |  |  else
13 |  |  |  nonPCPs ← nonPCPs ∪ {c}
14 |  |  end
15 |  |  newCs ← generate_direct_subsets(c)
16 |  |  Cands ← Cands ∪ newCs
17 |  end
18 end
```

initial candidate set for mining (step 3). When the set is not empty, Algorithm 1 pops a candidate from the set (step 5) and first checks whether the candidate has been processed (step 6). If not, it collects the participating instances of each feature of this candidate in the hash structure and computes its PI value (steps 7–8). The minimum prevalence threshold of this candidate is also computed subsequently (step 9). Algorithm 1 determines the prevalence of this candidate (step 10) and then puts it into the PCP set (step 11) or the nonPCP set (step 13). Finally, the direct subsets of this candidate are generated as new candidates and put into the initial candidate set (steps 15–16).

4 Experiments and Analysis

This section performs a series of experimental verifications. We selected three representative algorithms for mining PCPs. Joinless is a typical algorithm that adopts the level-wise search mining framework [15]. It searches PCPs from size $k = 2$ and uses the downward closure property of PI to prune unnecessary high-size candidates. IDS is a typical algorithm that adopts the clique-based mining framework [1]. It is similar to the method in this work, but since it generates a lot of cliques, too many repetitive neighbor relationships are stored in a two-level hash structure, and it becomes expensive to search for candidate participating instances. At the same time, the two-layer hash structure also increases the memory requirement. Coloc is a representative of the direct mining method [14].

It first generates all candidates using a tree similar to FP-tree growth, and then tests the prevalence of these candidates. All algorithms are implemented in C++ and run on a computer with Intel(R) Core i7-3770 and 16GB memory.

4.1 Datasets

There are four real datasets in our experiment. The first dataset, Las Vegas, is extracted from the Yelp dataset[2], which include different types of business information in the United States. The remaining three datasets are POI data collected from Shanghai [9], Beijing [1], and California (See footnote 1), respectively. The real datasets are preprocessed (e.g., removing noise, outliers, etc.) before mining. At the same time, a synthetic data generator [15] is also used to generate a series of synthetic datasets. Table 2 lists the basic parameters of these datasets.

Table 2. A summary of the datasets in the experiments.

Name	Spatial size	# features	# instances
Real datasets			
Las Vegas	38,649× 62,985 km^2	17	22,725
Shanghai	65,557 × 113,690 km^2	15	41,454
Beijing	138.7 × 227 km^2	19	104,909
California	845.2 × 1152.1 km^2	18	92,726
Synthetic datasets			
Figs. 4(a), 5(a), 6(a), 7(a), 8(a), 9(a)	5k × 5k	15	50k
Figs. 10(a), 11(a)	5k × 5k	15	*

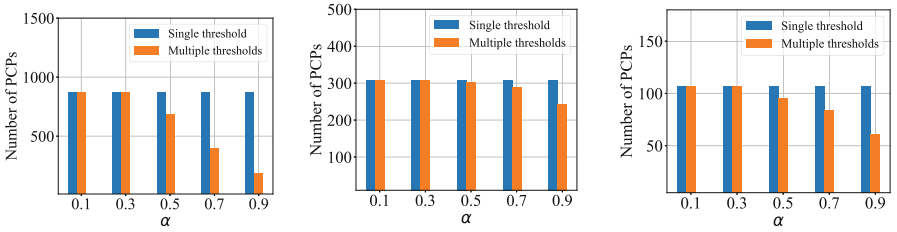
1k = 1000, * : variable

Moreover, the minimum prevalence threshold of each feature is automatically generated as given by the following equation:

$$\mu(f_t) = max\{\alpha \times \frac{|I(f_t)|}{|I_{max}|}, \mu\}. \tag{2}$$

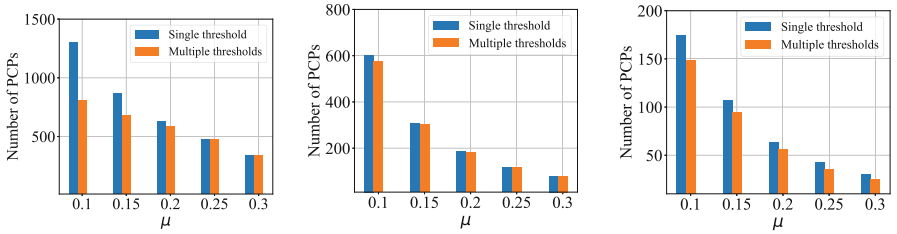
where $1 \geq \alpha \geq 0$ is the constant that is used to control the minimum prevalence threshold of a feature as a function of its number of instances, while $|I(f_t)|$ and $|I_{max}|$ are the number of instances of feature f_t and the maximal number of instances of the feature, respectively. The parameter μ is the user-specified least minimum prevalence threshold (same as the single minimum threshold in traditional PCP mining). When $\alpha = 0$, then a single threshold is used to all features and this will be equivalent to traditional PCP mining.

[2] https://www.yelp.com/dataset.



(a) Synthetic ($d = 40$) **(b)** Las Vegas ($d = 160$m) **(c)** Shanghai ($d = 300$m)

Fig. 4. Numbers of PCPs discovered in different values of α ($\mu = 0.15$ for all).



(a) Synthetic ($d = 40$) **(b)** Las Vegas ($d = 160$m) **(c)** Shanghai ($d = 300$m)

Fig. 5. Numbers of PCPs discovered in different least minimum prevalence thresholds ($\alpha = 0.5$ for all).

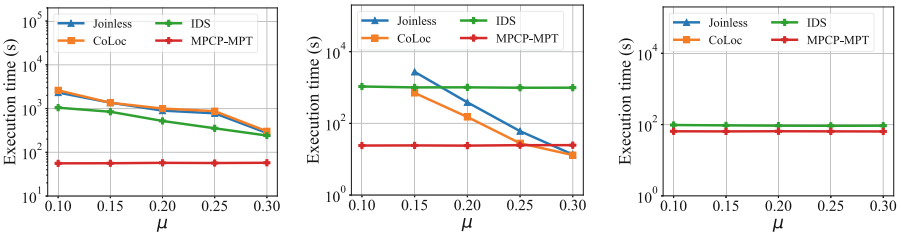
4.2 Examining in Numbers of PCPs Discovered

This experiment investigates the difference in mining results when using a single and multiple prevalence thresholds. Figure 4 records the number of PCPs mined when changing α, i.e., changing the threshold of each feature. It can be seen that when α takes a large value, the threshold of each feature differs more and more, and at the same time, the difference with the least minimum prevalence threshold, μ, specified by users is large, thus the number of PCPs discovered by using multiple prevalence thresholds is less than the traditional single prevalence threshold. This shows that when a single prevalence threshold is used, numerous PCPs are found but few of them are selected by considering the distinct importance of each feature in datasets and many redundant patterns are discovered. When α takes a small value, according to Eq. 2, the threshold of each feature tends to μ, thus the results between using a single threshold and multiple thresholds are not obvious, that is, at this time, both features and patterns use a same threshold, μ, and it just likes traditional mining.

The number of PCPs discovered when changing the least minimum prevalence threshold, μ, is depicted in Fig. 5. It can be seen that when μ is increased, the threshold of each feature will be close to μ, so the mining results of using a single threshold and multiple thresholds are close. When μ is small, the difference between the thresholds of features and the least minimum prevalence threshold is widened, so multiple prevalence thresholds can better reflect its effect.

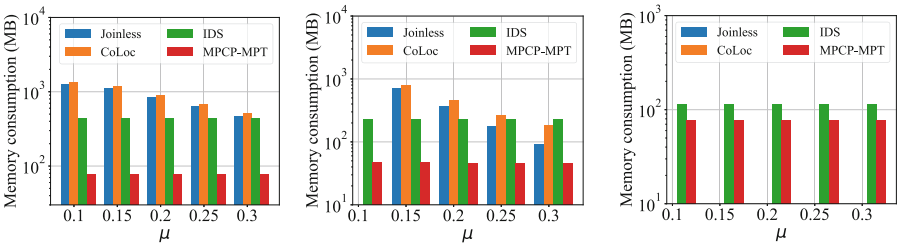
4.3 Examining in the Mining Efficiency

Different Least Minimum Prevalence Thresholds: Figures 6 and 7 show the execution time and memory consumption of the compared algorithms under different datasets as the least minimum prevalence threshold changes. It can be seen that as μ increases, the execution time and memory consumption of Joinless and CoLoc decrease, because the high-size candidates are pruned more effectively. However, interesting patterns are easily lost under large prevalence thresholds. As μ changes, the execution time and memory consumption of IDS do not change much, because it searches candidates from the longest size to the lowest ($k = 2$) size, thus the pruning effect is not good. The algorithm designed in this paper, MPCP-MPT, gives better results than the other three algorithms.



(a) Synthetic ($d = 40$) **(b)** Las Vegas ($d = 160$m) **(c)** Shanghai ($d = 300$m)

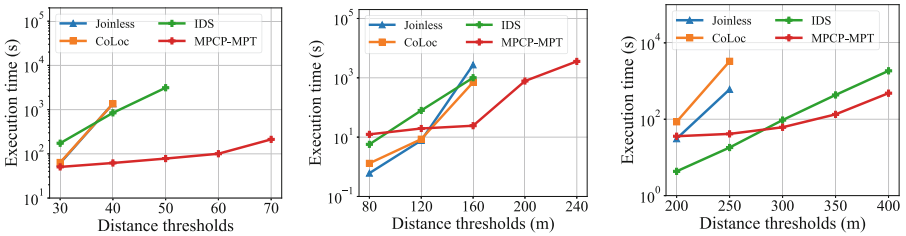
Fig. 6. The execution time of the compared algorithms in different values of least minimum prevalence thresholds ($\alpha = 0.5$).



(a) Synthetic ($d = 40$) **(b)** Las Vegas ($d = 160$m) **(c)** Shanghai ($d = 300$m)

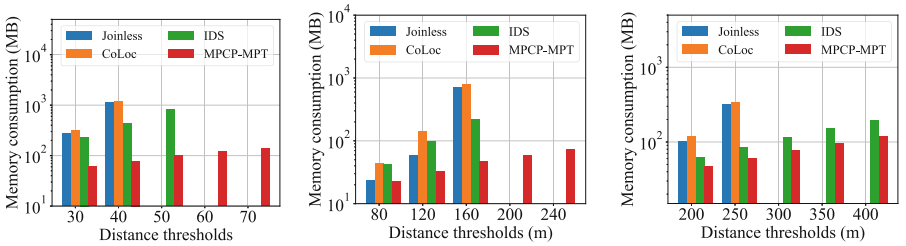
Fig. 7. The memory consumption of the compared algorithms in different values of least minimum prevalence thresholds ($\alpha = 0.5$).

Different Distance Thresholds: The execution time and memory consumption of the four algorithms at different distance thresholds are recorded in Figs. 8 and 9. It can be seen that as the distance threshold increases, the number of neighboring relationships increases, and the number of co-location instances of each candidate also increases, the performance of Joinless, CoLoc and IDS deteriorates quickly. On the contrary, MPCP-MPT can show better performance.



(a) Synthetic **(b)** Las Vegas **(c)** Shanghai

Fig. 8. The execution time of the compared algorithms in different values of distance thresholds ($\mu = 0.15$ for all and $\alpha = 0.5$).

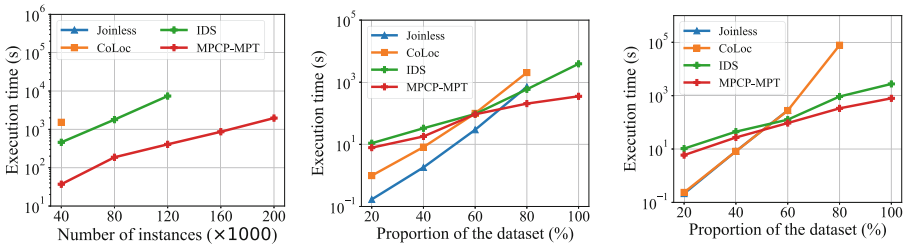


(a) Synthetic **(b)** Las Vegas **(c)** Shanghai

Fig. 9. The memory consumption of the compared algorithms in different values of distance thresholds ($\mu = 0.15$ for all and $\alpha = 0.5$).

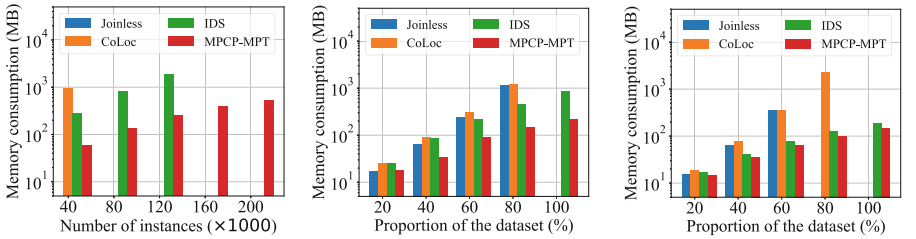
4.4 Examining in the Scalability

Figures 10 and 11 record the execution time and memory consumption of the four algorithms in different sizes of datasets. In order to produce different sizes of the real datasets, we sampled Beijing and California at different proportions. It can be seen that as the dataset size increases, the execution time and memory consumption of the four algorithms increase accordingly, but the performance of Joinless, CoLoc, and IDS deteriorates quickly, while the designed MPCP-MPT can provide better scalability.



(a) Synthetic ($d = 35$) **(b)** Beijing ($d = 200$m) **(c)** California ($d = 2000$m)

Fig. 10. The execution time of the compared algorithms in different sizes of datasets ($\mu = 0.15$ for all and $\alpha = 0.5$).



(a) Synthetic ($d = 35$) **(b)** Beijing ($d = 200$m) **(c)** California ($d = 2000$m)

Fig. 11. The memory consumption of the compared algorithms in different sizes of datasets ($\mu = 0.15$ for all and $\alpha = 0.5$).

5 Conclusion

In this work, we introduce mining PCPs using multiple minimum prevalence thresholds, that is, each feature is assigned a threshold to effectively reflect the differences between features in a pattern and the differences between patterns so that the mining results are more meaningful. To efficiently mine PCPs using multiple minimum prevalence thresholds, a mining algorithm based on maximal clique and hash structure is designed. This algorithm avoids the traditional mining algorithm idea (i.e., generating candidates first, finding co-location instances of the candidates, and then collecting participating instances), it directly collects participating instances of candidates. Experimental results on synthetic and real datasets with different parameters in different scenarios show that the designed method is more effective and efficient than existing algorithms using a single minimum prevalence threshold.

References

1. Bao, X., Wang, L.: A clique-based approach for co-location pattern mining. Inf. Sci. **490**, 244–264 (2019)

2. Chen, Y., Gan, W., Wu, Y., Philip, S.Y.: Privacy-preserving federated mining of frequent itemsets. Inf. Sci. **625**, 504–520 (2023)
3. Eppstein, D., Löffler, M., Strash, D.: Listing all maximal cliques in sparse graphs in near-optimal time. In: Cheong, O., Chwa, K.-Y., Park, K. (eds.) ISAAC 2010. LNCS, vol. 6506, pp. 403–414. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-17517-6_36
4. He, Z., Deng, M., Xie, Z.: Discovering the joint influence of urban facilities on crime occurrence using spatial co-location pattern mining. Cities **99**, 102612 (2020)
5. Huang, Y., Shekhar, S., Xiong, H.: Discovering colocation patterns from spatial data sets: a general approach. IEEE Trans. Knowl. Data Eng. **16**(12), 1472–1485 (2004)
6. Sainju, A.M., Aghajarian, D., Jiang, Z.: Parallel grid-based colocation mining algorithms on GPUs for big spatial event data. IEEE Trans. Big Data **6**(1), 107–118 (2018)
7. Sheshikala, M., Rao, D.R.: A map-reduce framework for finding clusters of colocation patterns-a summary of results. In: IACC, pp. 129–131. IEEE (2017)
8. Tran, V.: Meta-PCP: a concise representation of prevalent co-location patterns discovered from spatial data. Expert Syst. Appl. **213**, 119255 (2023)
9. Tran, V., Wang, L., Chen, H., Xiao, Q.: MCHT: a maximal clique and hash table-based maximal prevalent co-location pattern mining algorithm. Expert Syst. Appl. **175**, 114830 (2021)
10. Tran, V., Wang, L., Zhou, L.: A spatial co-location pattern mining framework insensitive to prevalence thresholds based on overlapping cliques. Distrib. Parallel Databases **41**(4), 511–548 (2023)
11. Wang, L., Bao, X., Zhou, L.: Redundancy reduction for prevalent co-location patterns. IEEE Trans. Knowl. Data Eng. **30**(1), 142–155 (2017)
12. Yao, X., Jiang, X., Wang, D., Yang, L., Peng, L., Chi, T.: Efficiently mining maximal co-locations in a spatial continuous field under directed road networks. Inf. Sci. **542**, 357–379 (2021)
13. Yoo, J.S., Boulware, D., Kimmey, D.: A parallel spatial co-location mining algorithm based on mapreduce. In: Big Data, pp. 25–31. IEEE (2014)
14. Yoo, J.S., Bow, M.: A framework for generating condensed co-location sets from spatial databases. Intell. Data Anal. **23**(2), 333–355 (2019)
15. Yoo, J.S., Shekhar, S.: A joinless approach for mining spatial colocation patterns. IEEE Trans. Knowl. Data Eng. **18**(10), 1323–1337 (2006)